# Task 1: Web App QA & Debug Report

**Role:** Automation & QA Developer Candidate

**Application Tested:** OrangeHRM (Open Source Demo)

## Bug Report

| # | Title / Summary | Steps to Reproduce | Expected vs Actual | Severity | Suspected Cause |
|---|---|---|---|---|---|
| 1 | First Name accepts numerical values | 1. Go to PIM > Add Employee.<br>2. Enter "0000" in First Name.<br>3. Click Save. | **Expected:** Form rejects input with a validation error.<br>**Actual:** Record saves successfully with numbers as a name. | Medium | Missing data type/regex validation on the frontend and backend. |
| 2 | Last Name accepts special characters | 1. Go to PIM > Add Employee.<br>2. Enter "08872@344" in Last Name.<br>3. Click Save. | **Expected:** Form rejects input containing symbols.<br>**Actual:** Record saves successfully. | Medium | Lack of character sanitization for standard text fields. |
| 3 | Employee ID accepts special characters | 1. Go to PIM > Add Employee.<br>2. Enter "11^^^^^^" in Employee ID.<br>3. Click Save. | **Expected:** Field restricts input to alphanumeric characters.<br>**Actual:** Record saves successfully. | Low | Input validation allows symbols instead of strictly alphanumeric. |
| 4 | Employee ID can be left blank | 1. Go to PIM > Add Employee.<br>2. Clear the autofilled Employee ID.<br>3. Click Save. | **Expected:** Validation error stating ID is required.<br>**Actual:** Employee is saved without an ID. | Medium | Field is not marked as mandatory in the form validation or database schema. |

| # | Title / Summary | Steps to Reproduce | Expected vs Actual | Severity | Suspected Cause |
|---|---|---|---|---|---|
| 5 | Unhandled Exception on HTML injection | 1. Go to PIM > Add Employee.<br>2. Enter HTML tags (<h1>Test</h1>) in Employee ID.<br>3. Click Save. | **Expected:** Clear validation error preventing HTML.<br>**Actual:** System throws generic "Unexpected Error Occurred" toast. | High | Backend fails to gracefully handle HTML payloads and throws a generic 500 error instead of a 400 Bad Request. |

## Root Cause Analysis: Unhandled Exception on HTML Injection (Bug #5)

The issue occurs when a user inputs HTML tags (such as `<h1>Test</h1>`) into the Employee ID field, resulting in a generic "Unexpected Error Occurred" toast notification. This happens because the application lacks proper input sanitization on the client-side and graceful error handling on the server-side. When the backend receives the HTML payload, the database or API likely rejects it due to security constraints (such as a WAF blocking potential XSS attempts) or a data-type mismatch. However, instead of catching this specific exception and returning a 400 Bad Request with a clear, actionable validation message, the server throws an unhandled exception or 500 Internal Server Error. To fix this, developers should implement strict regex validation on the frontend to only allow alphanumeric characters in the Employee ID field. Additionally, the backend must sanitize all incoming payloads and utilize proper `try/catch` blocks to ensure backend exceptions are mapped to readable validation errors for the end user.